

AI TRAINING – WEEK 2

DAY 4 ASSIGNMENT

OOP Basics: Classes, Objects, Constructors, Methods & Inheritance

Plan → Create Classes → Use Objects → Reuse Code

1. What You Will Learn

You will learn the basics of Object-Oriented Programming (OOP) in Python. You will create classes and objects, initialize object data using constructors, define methods, and understand how inheritance helps reuse code.

- Classes – define the structure and behavior of objects
- Objects – create individual instances of a class
- Constructors – initialize object attributes using `__init__()`
- Attributes – store information inside an object
- Methods – define actions that objects can perform
- Inheritance – create a child class from an existing parent class
- Use `self` to access the current object's attributes and methods
- Build a small class-based application

2. Student Deliverable – Keep It Simple

For this assignment, your main submission is ONE Python program that demonstrates the OOP concepts.

Main file: `oop_practice_YourName.py`

Keep the program and any supporting Python files in the same project folder.

- Create at least two classes

```
class practise:
    print("Hello")
class practise1:
    print("Hi")
obj=practise()
obj1=practise1()
```

- Create and use objects

```
class practise:
    print("Hello")
class practise1:
    print("Hi")
obj=practise()
obj1=practise1()
```

- Use constructors to initialize attributes

```

class practise:
    def __init__(self,name,age,course):
        self.name=name
        self.age=age
        self.course=course
    def display(self):
        print("Name:",self.name)
        print("Age:",self.age)
        print("course:",self.course)
text=practise("devendra",21,"cyber")
text.display()

```

- Write methods that perform useful actions

```

class Bank_manager:
    def __init__(self,name,account,ifsc,balance):
        self.name=name
        self.account=account
        self.ifsc=ifsc
        self.balance=balance
    def deposit(self,amount):
        if amount>0:
            self.balance+=amount
            return f"deposit_balance :{amount} , new_balance:{self.balance}"
        else:
            print("invalid balance")
    def withdraw(self,amount):
        if 0<amount<self.balance:
            self.balance-=amount
            print(f"withdraw amount :{amount},new_balance :{self.balance}")
        else:
            return "insufficient amount"
    def balance_enquiry(self):
        return f"account holder:{self.name}\n account
number:{self.account}\nIFSC:{self.ifsc}\nbalance:{self.balance}"

bank=Bank_manager("devendra",98764321,"i dont know",10)
print(bank.balance_enquiry())
print(bank.deposit(500))
print(bank.withdraw(260))
print(bank.balance_enquiry())

```

- Demonstrate simple inheritance

```

class person:
    def __init__(self,name):
        self.name=name
class bank_account(person):
    def __init__(self,name,account,balance):
        super().__init__(name)
        self.account=account
        self.balance=balance
    def details(self):
        print(f"name:{self.name}\naccount
number:{self.account}\nbalance:{self.balance}")
text=bank_account("devendra",9432345,10)
text.details()

```

- Test the program with different object values

```

class person:
    def __init__(self,name):
        self.name=name
class bank_account(person):
    def __init__(self,name,account,balance):
        super().__init__(name)
        self.account=account
        self.balance=balance
    def details(self):
        print(f"name:{self.name}\naccount
number:{self.account}\nbalance:${self.balance}")
text=bank_account("dinesh",9567432345,10000)
text.details()

```

- Use clear names and readable code

```

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
    def display_details(self):
        print("Student Name:", self.name)
        print("Student Marks:", self.marks)
student1 = Student("Devendra", 90)
student1.display_details()

```

No screenshots or separate Word/PDF submission is required unless your trainer asks for them.

3. Part A – Class and Object Basics

Create a class called Student. Give it a name and age. Create two Student objects with different values.

- Print each student's name and age.
- Change one object's name and print it again.
- Observe that changing one object does not automatically change the other object.

```
class students:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def details(self):
        print("name:", self.name)
        print("age:", self.age)

new = students("dini", 22)
new2 = students("devendra", 21)
new.details()
new2.details()
```

Example

```
class Student:
    pass

student1 = Student()
student2 = Student()

student1.name = "Rahul"
student1.age = 21
student2.name = "Priya"
student2.age = 22

print(student1.name, student1.age)
print(student2.name, student2.age)
```

Task: Rewrite the example so name and age are initialized through a constructor instead of being assigned after object creation.

4. Part B – Constructors and Attributes

A constructor is commonly written using `__init__()`. It runs when an object is created and can assign initial values to that object.

- Create a class called Mobile with brand, model, and price attributes.
- Use `__init__()` to initialize all three attributes.
- Create at least three Mobile objects with different values.
- Print the details of every object.

Example

```
class Mobile:
    def __init__(self, brand, model, price):
        self.brand = brand
        self.model = model
```

```
self.price = price
```

```
phone1 = Mobile("Samsung", "A55", 35000)  
phone2 = Mobile("Apple", "iPhone 15", 65000)
```

```
print(phone1.brand, phone1.model, phone1.price)  
print(phone2.brand, phone2.model, phone2.price)
```

- Add a new attribute called storage and display it.
- Try creating an object without passing a required value. Observe the error and explain why it happens.

```
class Mobile:  
    def __init__(self, brand, model, price, storage):  
        self.brand = brand  
        self.model = model  
        self.price = price  
        self.storage = storage  
    def display(self):  
        print(f"brand:{self.brand}\nmodel:{self.model}\nprice:{self.price}\nstorage:{self.storage}")  
phone1 = Mobile("Samsung", "A55", 35000, "64GB")  
phone2 = Mobile("Apple", "iPhone 15", 65000, "128GB")  
phone1.display()  
phone2.display()
```

5. Part C – Methods

Methods are functions defined inside a class. They commonly use self to access the current object's attributes.

- Create a class called BankAccount with account_holder and balance.
- Add a display_balance() method.
- Add a deposit(amount) method that increases the balance.
- Add a withdraw(amount) method that decreases the balance only when enough money is available.
- Create an object and test deposit and withdrawal operations.

Example

```
class BankAccount:  
    def __init__(self, account_holder, balance):  
        self.account_holder = account_holder  
        self.balance = balance  
  
    def display_balance(self):  
        print("Account holder:", self.account_holder)  
        print("Balance:", self.balance)  
  
    def deposit(self, amount):  
        self.balance += amount  
  
account = BankAccount("Rahul", 1000)  
account.display_balance()  
account.deposit(500)  
account.display_balance()
```

Challenge: Add validation so a negative deposit is rejected. Also handle a withdrawal request that is greater than the current balance.

```
class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self.balance = balance

    def display_balance(self):
        print("Account holder:", self.account_holder)
        print("Balance:", self.balance)

    def deposit(self, amount):
        self.balance += amount
        print("Deposited:", amount)

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print("Withdrawn:", amount)
        else:
            print("Insufficient balance!")

account = BankAccount("Rahul", 1000)
account.display_balance()
account.deposit(500)
account.display_balance()
account.withdraw(200)
account.display_balance()
account.withdraw(2000)
account.display_balance()
```

6. Part D – Methods That Return Values

A method can print a result or return a value using return. Practice returning values from methods.

- Create a Rectangle class with length and width.
- Add an area() method that returns length multiplied by width.
- Add a perimeter() method that returns 2 multiplied by the sum of length and width.
- Create two Rectangle objects and print their areas and perimeters.

Example

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
```

```

def area(self):
    return self.length * self.width

def perimeter(self):
    return 2 * (self.length + self.width)

shape = Rectangle(10, 5)
print("Area:", shape.area())
print("Perimeter:", shape.perimeter())

```

```

class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def area(self):
        return self.length * self.width
    def perimeter(self):
        return 2 * (self.length + self.width)
shape1 = Rectangle(10, 5)
shape2 = Rectangle(8, 4)

print("Rectangle 1")
print("Area:", shape1.area())
print("Perimeter:", shape1.perimeter())

print("Rectangle 2")
print("Area:", shape2.area())
print("Perimeter:", shape2.perimeter())

```

7. Part E – Inheritance Overview

Inheritance allows a child class to reuse attributes and methods from a parent class. A child class can also define its own methods.

- Create a parent class called Person with name and age.
- Add a introduce() method that prints the person's details.
- Create a child class called Employee that inherits from Person.
- Add an employee_id attribute to Employee.
- Create an Employee object and call both the inherited method and a method of its own.

Example

```

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print("Name:", self.name)
        print("Age:", self.age)

class Employee(Person):

```

```

def __init__(self, name, age, employee_id):
    super().__init__(name, age)
    self.employee_id = employee_id

def show_employee_id(self):
    print("Employee ID:", self.employee_id)

employee = Employee("Ananya", 23, "E101")
employee.introduce()
employee.show_employee_id()

```

```

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def introduce(self):
        print("Name:", self.name)
        print("Age:", self.age)
class Employee(Person):
    def __init__(self, name, age, employee_id):
        super().__init__(name, age)
        self.employee_id = employee_id

    def show_employee_id(self):
        print("Employee ID:", self.employee_id)

employee = Employee("Ananya", 23, "E101")

employee.introduce()
employee.show_employee_id()

```

Tasks:

- Explain what Person and Employee represent in this example.
- Explain why `super().__init__(name, age)` is used.
- Create another child class called Trainer with a subject attribute.
- Try calling a method that exists only in the child class from a parent-class object. Record what happens.

8. Part F – Organizing OOP Code into Modules

A module is a Python file that can contain classes and functions. Practice placing a class in one file and importing it into another.

Suggested structure:

```

oop_project/
  main.py
  student.py

```

- In `student.py`, create a `Student` class with a constructor and a `display_details()` method.
- In `main.py`, import `Student` from `student.py`.
- Create at least two `Student` objects and call `display_details()` for each.

```
# student.py
class Student:
    def __init__(self, name, course):
        self.name = name
        self.course = course

    def display_details(self):
        print("Name:", self.name)
        print("Course:", self.course)

# main.py
from student import Student

student1 = Student("Rahul", "Python")
student2 = Student("Priya", "Cyber Security")

student1.display_details()
student2.display_details()
```

Keep both files in the same folder for this basic example.

Output:

Name: Rahul

Course: Python

Name: Priya

Course: Cyber Security

9. Part G – Quick Debugging

Find the problem, fix it, run the code, and explain the mistake.

Debugging Problem 1

```
class Student:
    def __init__(self, name):
        name = name

student = Student("Rahul")
print(student.name)
```

Task: Why does this code fail when printing student.name? Fix the constructor.

```
class Student:
    def __init__(self, name):
        self.name = name

student = Student("Rahul")
print(student.name)
```

we need use self while initializing the variables.

Debugging Problem 2

```
class Car:
    def __init__(self, brand):
        self.brand = brand

car = Car()
print(car.brand)
```

Task: Identify the problem and fix the object creation or constructor.

```
class Car:
    def __init__(self, brand):
        self.brand = brand

car = Car("volvo")
print(car.brand)
```

we need to give values while creating the objects.

Debugging Problem 3

```
class Calculator:
    def add(a, b):
        return a + b

calc = Calculator()
print(calc.add(5, 3))
```

Task: Explain the role of self in an instance method and correct the code.

```
class Calculator:
    def add(self, a, b):
        return a + b

calc = Calculator()
print(calc.add(2, 3))
```

we need to give self variable inside a method.

Debugging Problem 4

```
class Animal:
    def speak(self):
        print("Animal sound")

class Dog:
    def bark(self):
        print("Woof")

dog = Dog()
dog.speak()
```

Task: The code attempts to call a method that Dog does not have. Use inheritance or change the method call to make it work.

```
class Animal:
    def speak(self):
        print("Animal sound")

class Dog(Animal):
    def bark(self):
        print("Woof")

dog = Dog()
dog.speak()
dog.bark()
```

10. Part H – Main Hands-on Activity: Student Record System

Build a simple class-based Student Record System. The program should create student objects, store their details, and display useful information.

Each student should have:

- name
- roll_number
- marks

Your program should:

- Create a Student class with a constructor
- Add a display_details() method
- Add a result() method that returns Pass if marks are 40 or above and Fail otherwise
- Create at least four student objects with different marks
- Display each student's details and result
- Find and display the student with the highest marks
- Handle or prevent invalid marks outside the range 0–100
- Use a list to store the student objects

Starter example

```
class Student:
    def __init__(self, name, roll_number, marks):
        self.name = name
        self.roll_number = roll_number
        self.marks = marks

    def display_details(self):
        print(self.name, self.roll_number, self.marks)

    def result(self):
        if self.marks >= 40:
            return "Pass"
        return "Fail"

students = [
    Student("Rahul", 1, 85),
    Student("Priya", 2, 92),
    Student("Arjun", 3, 38),
    Student("Ananya", 4, 76)
]

for student in students:
    student.display_details()
    print("Result:", student.result())
```

Extend the starter example to validate marks and find the highest scorer. You may create helper methods if they make the program clearer.

Bonus: Create a GraduateStudent class that inherits from Student and adds a specialization attribute.

```
class Student:
    def __init__(self, name, roll_number, marks):
        if marks < 0 or marks > 100:
            raise ValueError("Marks must be between 0 and 100")
```

```

        self.name = name
        self.roll_number = roll_number
        self.marks = marks

    def display_details(self):
        print("Name:", self.name)
        print("Roll Number:", self.roll_number)
        print("Marks:", self.marks)

    def result(self):
        if self.marks >= 40:
            return "Pass"
        else:
            return "Fail"
students = [
    Student("Rahul", 1, 85),
    Student("Priya", 2, 92),
    Student("Arjun", 3, 38),
    Student("Ananya", 4, 76)
]
for student in students:
    student.display_details()
    print("Result:", student.result())
    print("-----")
topper = max(students, key=lambda student: student.marks)
print("Highest Scorer:")
topper.display_details()
print("Result:", topper.result())

```

11. Suggested Program Structure

- Student – stores student details and provides student-related methods
- display_details() – displays one student's information
- result() – returns Pass or Fail
- find_highest_scorer(students) – identifies the student with the highest marks
- main() – creates objects and controls the program

Bonus: Put the Student class in student.py and import it into main.py.

12. Part I – Explain in Your Own Words

1. What is the difference between a class and an object?

Ans: OOP is a way of writing programs using classes and objects to organize and reuse code.

2. What is the purpose of `__init__()`?
Ans: It is used to initialize an object's data when it is created.
3. What does `self` refer to inside an instance method?
Ans: `self` refers to the current object.
4. What is an attribute?
Ans: An attribute is a variable that belongs to an object or class.
5. What is a method?
Ans: A method is a function defined inside a class.
6. What is inheritance?
Ans: Inheritance allows a child class to reuse properties and methods of a parent class.
7. Why do we use `super()` in a child class?
Ans: We use `super()` to access methods of the parent class.
8. What is the difference between `print()` and `return` in a method?
Ans: `print()` displays a value, while `return` sends a value back to the caller.
9. Why can classes make a program easier to organize?
Ans: Classes help group related data and methods together, making code easier to manage and reuse.

13. Daily Deliverable

Submit your working OOP practice program.

File name: `oop_practice_YourName.py`

Your program should demonstrate:

- Classes and objects
- Constructors and attributes
- Instance methods
- Methods that return values
- Simple inheritance
- Object creation and method calls
- Readable code and basic validation

14. Student Submission Checklist

- ☐ can explain the difference between a class and an object.
- ☐ created objects from a class.
- ☐ used `__init__()` to initialize attributes.
- ☐ used `self` correctly in instance methods.
- ☐ wrote methods that print or return results.
- ☐ understand basic inheritance.
- ☐ used `super()` in a child class.
- ☐ tested my code with different values.
- ☐ fixed the debugging examples.
- ☐ can explain the main OOP concepts in my own words.

15. After This Assignment, You Should Be Able To

- Describe the purpose of OOP in Python.
- Create classes and instantiate objects.

- Initialize object attributes using constructors.
- Write methods that use object data.
- Return calculated values from methods.
- Understand and implement simple inheritance.
- Import a class from another Python module.
- Build a small class-based application.

Key Principle: A class defines a blueprint; objects are individual instances that use its attributes and methods.